UNIVERSITY OF PERADENIYA

DEPARTMENT OF STATISTICS AND COMPUTER SCIENCE

END SEMESTER EXAMINATION - SEMESTER II (2022/2023)

CSC1051 - PROGRAMMING LABORATORY II

Answer **ALL** Questions Time Allowed: **3** Hours

Create a folder named CSC1051_End_S21XXX in your H drive. Create a new Java project named S21XXX. Create two packages named Q1 and Q2 inside the created Java project. Implement the answer for question 1 and question 2 inside Q1 and Q2 packages respectively. Save all your work inside the CSC1051_End_S21XXX folder created in your H drive. Only the files stored in this folder will be considered grading.

Question 1 [70 Marks]

You are tasked with developing a system that facilitates connections between freelancers and clients for effective project management.

You are required to create constructors for each class with necessary parameters. Marks will be allocated for following OOP concepts and best programming practices.

a) Create an **abstract class Project** with the following attributes and methods:

Attributes:

- **String projectName:** The name of the project.
- **String clientName:** The client associated with the project.
- **double budget:** The budget allocated for the project.
- **String status:** All new projects are created with the status as "In Progress".
- **static int totalProjects:** Tracks the total number of projects created. (Incremented automatically when a new project is instantiated)

Methods:

- Create an **abstract** method **double calculateTotalCost()** to return the total project cost.
- Create a static method **int getTotalProjects()** to returns the total number of projects.

b) Create three subclasses of **Project** named as **DesignProject, DevelopmentProject,** and **MarketingProject**.

DesignProject Class

Attributes:

- **int hoursWorked:** Number of hours spent working on the project.
- **double hourlyRate:** Hourly rate for the work.
- **int revisions:** The number of design revisions requested by the client.
- **double revisionCost:** Cost per revision requested.

Methods:

- Override the method **calculateTotalCost()** which returns total cost as:

  Total Cost = (hoursWorked * hourlyRate) + (revisions * revisionCost)

- Create a method **boolean isOverBudget()** to return true if the total cost exceeds the budget.
- Create a method **boolean setRevisions()** that updates the number of revisions. Ensure the method validates input to allow a maximum of **5** revisions and return true if successful.

DevelopmentProject Class

Attributes:

- **int hoursWorked:** Number of hours spent working on the project.
- **double hourlyRate:** Hourly rate for the work.
- **double additionalCosts:** Costs for tools, software, or hardware.
- **boolean requiresTesting:** Indicates whether the project requires dedicated testing.

Methods:

- Override the method **calculateTotalCost()** which returns total cost as:

  Total Cost = (hoursWorked × hourlyRate) + additionalCosts

- Create a method **boolean isOverBudget()** to return true if the total cost exceeds the budget.
- Implement the method **String suggestOptimization()** to return a suggestion for optimizing costs.
  - If **hoursWorked > 100** and cost per hour exceeds **120%** of hourlyRate, return, "Add resources to improve efficiency." Cost per hour is given by:

    $$costPerHour = \frac{(calculateTotalCost - additionalCosts)}{hoursWorked}$$

  - If **additionalCosts > budget * 0.3**, return, "Re-evaluate non-essential expenditures."
  - If **requiresTesting** but **hoursWorked < 40**, return, "Increase resources for development."
  - Otherwise, return, “Development is on track. Keep up the current pace."

MarketingProject Class

Attributes:

- **double campaignCost:** The cost of the marketing campaign.
- **double additionalExpenses**: Any additional expenses incurred.

Methods:

- Override method **calculateTotalCost()** to return ***campaignCost + additionalExpenses***.

c) Define an interface **Billable** which should be implemented by **DesignProject** and **DevelopmentProject** and update with the following methods and attributes.

Attributes:

- **double TAX_RATE** = 0.18: A **constant** representing the tax rate.

Methods:

- Create **double calculateFinalAmount()** method to return the final amount to bill using following formula:

  Final amount = calculateTotalCost() + calculateTotalCost() * TAX_RATE

d) Create a **ProjectManager** class with following information.

Attributes:

- **Project[] projectList**: An array to store all projects in the system. Array length is 10.

Methods:

- Create **void addProject(Project project)** method to add a single project to the **projectList**.
- Create **void displayProjectDetails()** method to display details of all projects (name, client, budget, and status).

e) Create a **Main** Class to Simulate Project Management System

Inside the **main** method, perform the following operations and obtain the output provided in **Figure 1**.

1. Create two **DesignProject**s with following details,

Project Name	**Client Name**	**Budget**	**Additional Values**
Website Redesign	Client A	5000	Hours Worked: 80, Hourly Rate: 50, Revisions: 3, Revision Cost: 100
Logo Design	Client D	20000	Hours Worked: 30, Hourly Rate: 60, Revisions: 1, Revision Cost: 150

   - Change the status of the project “Logo Design” as "Completed".
   - Call the **calculateFinalAmount** method for project “Logo Design” and print the results.

2. Create a **DevelopmentProject** with following details and call **suggestOptimization()** and print the output.

Project Name	**Client Name**	**Budget**	**Additional Values**
Mobile App	Client B	10000	Hours Worked: 150, Hourly Rate: 75, Additional Costs: 2000, Requires Testing: true

3. Create a **MarketingProject** with following details,

Project Name	**Client Name**	**Budget**	**Additional Value**
Ad Campaign	Client C	7000	Campaign Cost: 4000, Additional Expenses: 500

4. Create an instance of the **ProjectManager** class. Add the projects you created using **addProject**.
5. Call the **displayProjectDetails** method.
6. Print the total number of projects created.

Expected Output of the main method:

```
Final Amount for the project Logo Design : 1950.0
Optimizations for the project Mobile App : Development is on track. Keep up the current pace.

Project Details
Project Name: Website Redesign, Client: Client A, Budget: 5000.0, Status: In Progress
Project Name: Logo Design, Client: Client D, Budget: 20000.0, Status: Completed
Project Name: Mobile App, Client: Client B, Budget: 10000.0, Status: In Progress
Project Name: Ad Campaign, Client: Client C, Budget: 7000.0, Status: In Progress

Total Number of Projects : 4
```

Figure 1

Question 2 [30 Marks]

You are tasked with developing a fitness tracking application that logs user workouts. The program will process workout data and allow for file writing to store the summary of workouts.

a) Create a **WorkoutLog** class with the following attributes and methods:

Attributes:

- **String date:** Date of the workout (in the format "yyyy-MM-dd").
- **String exerciseType:** Type of exercise
- **int duration:** Duration of the workout in minutes.
- **int caloriesBurned:** Calories burned during the workout.

Methods:

- Override the **toString()** method to return workout details in the following format:

```
"Date: 2024-11-01, Exercise: Running, Duration: 30 minutes, Calories Burned: 300"
```

Figure 2

b) Create a **FitnessTracker** class that handles workout data and file writing.

Attributes:

- **WorkoutLog[] logs**: An array of **WorkoutLog** objects with size of **10** entries.
- **int count**: The number of workout entries currently stored in the logs.

Methods:

- Create **void addWorkoutLog(WorkoutLog log)** method to add a new workout entry to the logs.
- Create **int calculateTotalWorkoutTime()** method to calculate and return the total workout time for all logged workouts.
- Create **int calculateTotalCaloriesBurned()** method to calculate and return the total calories burned for all logged workouts.
- Create **void writeWorkoutSummaryToFile()** method to take a **filePath** as paramenter and write all workout details to a file including a summary of total workout time and total calories burned followed by the details of each single workout log as given in **Figure 4**.

c) Create a **Main** class to implement the following.

1. **In the main method:**
    a. Create a **FitnessTracker** object.
    b. Prompt the user to enter workout data for multiple workouts. Ensure that the user can keep adding workouts until they choose to stop by entering “no”. (See ***Figure 3*** for an example user interaction)
    c. For each workout data create a **WorkoutLog** and call **addWorkoutLog()** to add that WorkoutLog to the **logs** array.
    d. After the user finishes entering data, write the workout summary to a file named ***workout_summary.txt*** inside the **CSC1051_End_S21XXX** folder located in your **H drive (See Figure 4).**

Example Interaction:

```
Enter Workout Date (yyyy-MM-dd): 2024-11-01
Enter Exercise Type (Running/Cycling/Yoga): Running
Enter Duration (minutes): 30
Enter Calories Burned: 300

Do you want to add another workout? (yes/no): yes

Enter Workout Date (yyyy-MM-dd): 2024-11-02
Enter Exercise Type (Running/Cycling/Yoga): Cycling
Enter Duration (minutes): 45
Enter Calories Burned: 400

Do you want to add another workout? (yes/no): no
```

Figure 3

Generated Text File *workout_summary.txt*:

```
Workout Summary
Total Workout Time: 75 minutes
Total Calories Burned: 700

Workouts:
Date: 2024-11-01, Exercise: Running, Duration: 30 minutes, Calories Burned: 300
Date: 2024-11-02, Exercise: Cycling, Duration: 45 minutes, Calories Burned: 400
```

Figure 4

End of Paper